# Algebraic Cipher Types: Confidentiality Trade-offs in Type Constructors over Trapdoor Computing

Alexander Towell
lex@metafunctor.com

March 2026

**Abstract**

A cipher map is a total function on bit strings that implements a trapdoor approximation of a latent function: the untrusted machine evaluates it blindly; only the trusted machine, holding the secret, can encode inputs and decode outputs. We study what happens when algebraic type constructors—products, sums, and exponentials—are applied to cipher values. Each constructor forces a specific confidentiality/functionality trade-off that is absent in plaintext programming. Our main result is a sum-type impossibility: for $A + B$, either the tag (which variant) is hidden and pattern matching requires the trapdoor, or pattern matching is available to the untrusted machine and the tag is leaked. No encoding achieves both simultaneously. We introduce the *orbit closure* of a cipher value under a set of available operations and prove that confidentiality is bounded by the fraction of the cipher space reachable from any single value. Finally, we define a cipher Turing machine whose space complexity is $O(|Q| \cdot |\Gamma|)$, independent of the input domain size, contrasting with the $O(|X|)$ space of a cipher map—but at the cost of leaking head movement patterns.

## 1 Introduction

Trapdoor computing is a paradigm for evaluating functions on data whose meaning is hidden behind a one-way trapdoor (a cryptographic hash). A trusted machine encodes plaintext values into cipher values—opaque bit strings—and hands them, together with cipher maps (total functions on bit strings), to an untrusted machine. The untrusted machine evaluates the cipher maps and returns results. Without the trapdoor (the secret seed), it cannot decode the results or determine what function it computed. The formal framework of cipher maps, their four defining properties (totality, representation uniformity, correctness, composability), and three concrete constructions (HashSet, entropy map, trapdoor boolean algebra) are developed in [9].

Ordinary programming languages build complex types from simpler ones using a small set of type constructors: products ($A \times B$, pairs and records), sums ($A + B$, tagged unions and variants), and exponentials ($A \to B$, functions). In plaintext computing, these constructors are transparent: a product exposes both components, a sum carries a tag that enables pattern matching, and a function is called by providing an argument. When values are hidden behind a trapdoor, each of these operations introduces a confidentiality cost that has no plaintext analogue.

This paper identifies and formalizes those costs. The main contributions are:

1. **Sum-type impossibility** (Theorem 4.2). For cipher values of sum type $A+B$, tag hiding and pattern matching by the untrusted machine are mutually exclusive. This is an information-theoretic result, independent of the cipher map construction.

2. **Orbit closure and confidentiality bound** (Section 5). We define the orbit closure of a cipher value under available operations, prove monotonicity (Theorem 5.1), and derive an upper bound on confidentiality in terms of orbit size (Theorem 5.3).

3. **Cipher Turing machine** (Section 6). We exhibit a model of computation on cipher values whose space complexity is $O(|Q| \cdot |\Gamma|)$—depending on the program, not the data—but which leaks the head movement pattern.

We deliberately avoid game-based or simulation-based cryptographic definitions. Privacy in trapdoor computing comes from the one-way hash and the representation uniformity of cipher values, not from access-pattern indistinguishability. This is not ORAM [5], FHE [4], or garbled circuits [10]; it is a distinct paradigm where totality and noise closure provide the confidentiality guarantees.

## 2 Related Work

**Information flow type systems.** The sum-type impossibility (Theorem 4.2) is closely related to the implicit flow problem in language-based information security [7]: a conditional branch on a secret value leaks information through the control flow path taken. Our result formalizes this for cipher values: pattern matching on a cipher sum type is an implicit flow that leaks the tag. The difference is that information flow type systems prevent the leak at compile time, while cipher types quantify it at the representation level.

**Functional encryption.** Functional encryption [2] allows computing $f(x)$ on an encrypted $x$ by issuing function-specific decryption keys. Cipher maps differ in two ways: (1) they are information-theoretic (parameterized by $\eta$, $\varepsilon$, $\delta$) rather than game-based, and (2) they are approximate (the untrusted machine evaluates a total function with bounded error, not an exact decryption). The type-constructor trade-offs we identify (tag leakage for sums, correlation leakage for products) apply equally to functional encryption schemes that support algebraic operations.

**Searchable symmetric encryption.** SSE [3] enables keyword search on encrypted documents with simulation-based security guarantees. The Boolean search construction in Section 7.2 provides an alternative with information-theoretic parameters: cipher sets with cipher Boolean AND/OR/NOT, rather than encrypted indices with leakage profiles. Naveed et al. [6] demonstrated inference attacks on property-preserving encryption; the cipher map model avoids property preservation entirely (the untrusted machine cannot determine the output type from the cipher value).

## 3 Preliminaries

We recall the cipher map abstraction from [9]. All notation follows that paper; we cite rather than re-derive.

**Definition 3.1** (Cipher map [9, Def. 1.1])**.** A *cipher map* for a latent function $f : X \to Y$ is a tuple $(\hat{f}, \mathrm{enc}, \mathrm{dec}, s)$ where $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ is a total function on $n$-bit strings, $\mathrm{enc} : X \times \{0, \ldots, K-1\} \to \{0,1\}^n$ is an encoding function, $\mathrm{dec} : \{0,1\}^n \to Y \cup \{\perp\}$ is a decoding function, and $s$ is a secret (the trapdoor). The untrusted machine holds $\hat{f}$. The trusted machine holds enc, dec, and $s$.

A cipher map satisfies four properties, parameterized by $(\eta, \varepsilon, \mu, \delta)$ [9, Sec. 4]:

1. **Totality.** $\hat{f}$ is defined on all $2^n$ inputs. Out-of-domain outputs are indistinguishable from uniform under the random oracle model [1].
2. **Representation uniformity** ($\delta$-bounded). The marginal distribution of cipher values is $\delta$-close (in total variation distance) to uniform over $\{0,1\}^n$.
3. **Correctness** ($\eta$-bounded). For a random in-domain element $x$ and random representation index $k$, $\Pr[\mathrm{dec}(\hat{f}(\mathrm{enc}(x,k))) \neq f(x)] \leq \eta$.
4. **Composability.** For cipher maps $\hat{f}$ (correctness $\eta_f$) and $\hat{g}$ (correctness $\eta_g$), the composition $\hat{g} \circ \hat{f}$ has correctness $\eta_{g \circ f} = 1 - (1-\eta_f)(1-\eta_g)$.

The Bernoulli error model [8] provides the quantitative foundation: element-wise independence of errors reduces the composition analysis from exponentially many joint failure modes to per-element Bernoulli parameters. We use $h : \{0,1\}^* \to \{0,1\}^n$ throughout to denote a cryptographic hash modeled as a random oracle.

A cipher value $\mathrm{enc}(v,k)$ is a cipher map for the constant function $f_v(x) = v$; we use "cipher value" and "cipher map from a trivial domain" interchangeably.

**Definition 3.2** (Trusted and untrusted machines [9, Sec. 5])**.** The *trusted machine* $T$ holds the trapdoor $s$, enc, dec, and can encode, decode, and inject filler queries. The *untrusted machine* $U$ holds $\hat{f}$ and a collection of cipher values. $U$ can evaluate $\hat{f}$ on any bit string; it cannot decode, distinguish real from filler queries, or determine the domain $X$.

# 4 Cipher Type Constructors

We now examine how the standard algebraic type constructors interact with cipher values. For each constructor, there is a choice: apply the cipher operation to the entire constructed type, or construct from already-ciphered components. These two strategies yield different confidentiality and functionality properties. We write $\mathsf{C}(X)$ for the type of cipher values encoding elements of $X$.

## 4.1 Void and Unit

The void type $\mathbf{0}$ has no values. A cipher map $\hat{f}$ for a function with domain $\mathbf{0}$ is vacuously correct ($\eta = 0$) and carries no information—it is a total function on $\{0,1\}^n$ that is never queried on a valid encoding. From the untrusted machine's perspective, $\hat{f}$ is indistinguishable from a random function.

The unit type $\mathbf{1}$ has exactly one value, $\star$. A cipher value $\mathrm{enc}(\star, k) \in \{0,1\}^n$ encodes $\star$; with $K(\star) \geq 1$ representations, all cipher values are representations of the same underlying value. The untrusted machine knows that every valid cipher value encodes $\star$ but cannot distinguish valid encodings from noise (by totality). A cipher map $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ for a function from $\mathbf{1}$ is a cipher constant: it maps every valid encoding of $\star$ to a cipher encoding of $f(\star)$.

## 4.2 Product Types

The product type $A \times B$ consists of pairs $(a,b)$ with $a \in A$ and $b \in B$. In plaintext computing, a product comes with two projections $\pi_1 : A \times B \to A$ and $\pi_2 : A \times B \to B$. Under cipher encoding, we have two choices.

**Joint encoding: $\mathsf{C}(A \times B)$.** Encode the pair $(a, b)$ as a single cipher value $\mathrm{enc}((a, b), k) \in \{0, 1\}^n$. The untrusted machine sees one opaque bit string. It cannot determine $a$ or $b$ individually, nor can it detect correlations between $a$ and $b$. But the projections $\pi_1, \pi_2$ are not available: to extract $a$ from a cipher encoding of $(a, b)$, one would need a cipher map $\hat{\pi}_1 : \{0, 1\}^n \to \{0, 1\}^n$ such that $\mathrm{dec}_A(\hat{\pi}_1(\mathrm{enc}((a, b), k))) = a$—which requires a separate cipher map construction for each projection.

**Component-wise encoding: $\mathsf{C}(A) \times \mathsf{C}(B)$.** Encode $a$ and $b$ independently as $(\mathrm{enc}_A(a, k_1), \mathrm{enc}_B(b, k_2))$. The untrusted machine sees a pair of cipher values. Projections are trivially available (just take the first or second component). But the joint distribution of $(a, b)$ is no longer hidden: even if each component has marginal representation uniformity ($\delta \approx 0$), the correlation between $a$ and $b$ is preserved in the joint distribution of the cipher pair. This is the encoding granularity principle of [9, Sec. 9]: coarser encoding hides more correlations but blocks more operations.

**Proposition 4.1** (Product confidentiality trade-off)**.** *Let $A$ and $B$ be finite types.*

1. *Under joint encoding $\mathsf{C}(A \times B)$, the untrusted machine cannot distinguish $(a_1, b_1)$ from $(a_2, b_2)$ (up to $\delta$), but projections require a cipher map (and thus a separate construction by the trusted machine).*

2. *Under component-wise encoding $\mathsf{C}(A) \times \mathsf{C}(B)$, projections are free, but an adversary observing $N$ pairs $(\mathrm{enc}_A(a_i, k_{1i}), \mathrm{enc}_B(b_i, k_{2i}))$ can estimate the joint distribution of $(A, B)$ to arbitrary precision as $N \to \infty$.*

*Proof.* Part (1): Under joint encoding with representation uniformity $\delta$, each cipher value is $\delta$-close to uniform over $\{0, 1\}^n$. Without the trapdoor, the untrusted machine cannot decode. Projections require mapping a cipher encoding of $(a, b)$ to a cipher encoding of $a$; this is a function from the code space of $A \times B$ to the code space of $A$, which is exactly a cipher map and requires construction by the trusted machine.

Part (2): Under component-wise encoding, the $i$-th observation is $(\mathrm{enc}_A(a_i, k_{1i}), \mathrm{enc}_B(b_i, k_{2i}))$. Even with perfect marginal uniformity ($\delta = 0$), each $\mathrm{enc}_A$ is a deterministic function of $(a_i, k_{1i})$. If the same representation index is reused, repeated values produce identical cipher values; even without reuse, the number of distinct cipher values associated with each latent value is finite ($K(a)$ for each $a$). Given $N$ observations, the adversary builds the empirical joint distribution over cipher-value pairs. Since each cipher value maps to at most one latent value (given a fixed representation index), the empirical joint distribution of cipher pairs converges to a mixture of the latent joint distribution. By standard convergence results, $N = O(|A| \cdot |B|/\xi^2)$ observations suffice to estimate the joint to accuracy $\xi$ in total variation distance. □

## 4.3 Sum Types

The sum type $A + B$ is a tagged union: each value is either $\mathrm{inl}(a)$ for some $a \in A$, or $\mathrm{inr}(b)$ for some $b \in B$. The tag (left vs. right) enables pattern matching: given a value of type $A + B$, one inspects the tag to determine which handler to apply.

In plaintext computing, the tag is always available and pattern matching is free. Under cipher encoding, this is no longer the case.

**Theorem 4.2** (Sum-type impossibility)**.** *Let $A$ and $B$ be non-empty finite types with $|A| \geq 1$ and $|B| \geq 1$. For cipher values of sum type $A + B$, the following two properties are mutually exclusive:*

1. ***Tag hiding**: the untrusted machine cannot determine, from a cipher value alone, whether it encodes an $A$-value or a $B$-value with advantage better than $\delta$ (the representation uniformity parameter).*

2. ***Untrusted pattern matching**: the untrusted machine can route a cipher value to the correct handler ($A$-handler or $B$-handler) without assistance from the trusted machine.*

*Proof.* We prove the two directions by exhibiting the information-theoretic constraint.

**Joint encoding ($\mathsf{C}(A + B)$) achieves tag hiding but blocks pattern matching.** Under joint encoding, each value $v \in A + B$ (whether $\mathrm{inl}(a)$ or $\mathrm{inr}(b)$) is encoded as a single cipher value $\mathrm{enc}(v, k) \in \{0, 1\}^n$. Representation uniformity ($\delta$-bounded) guarantees that the marginal distribution of $\mathrm{enc}(v, k)$ over random $v$ and $k$ is $\delta$-close to uniform on $\{0, 1\}^n$. In particular, the set of cipher values encoding $A$-values and the set encoding $B$-values are not efficiently distinguishable by the untrusted machine: both are subsets of $\{0, 1\}^n$, and their union is $\delta$-close to covering $\{0, 1\}^n$ uniformly. Therefore the tag (which variant) is hidden up to $\delta$.

Now suppose the untrusted machine could nonetheless perform pattern matching: given $c = \mathrm{enc}(v, k)$, it could determine whether to apply the $A$-handler or the $B$-handler. This requires a function $\tau : \{0, 1\}^n \to \{A, B\}$ such that $\tau(\mathrm{enc}(\mathrm{inl}(a), k)) = A$ for all $a, k$ and $\tau(\mathrm{enc}(\mathrm{inr}(b), k)) = B$ for all $b, k$. But $\tau$ is exactly a distinguisher between $A$-encodings and $B$-encodings. Such a distinguisher exists only if the two sets of cipher values are disjoint (or nearly so) in a way detectable without the trapdoor. If $\tau$ succeeds with probability $1 - \gamma$, then the tag is leaked with advantage at least $1 - \gamma - \delta$, contradicting tag hiding when $\gamma$ is small.

**Component-wise encoding ($\mathsf{C}(A) + \mathsf{C}(B)$) permits pattern matching but leaks the tag.** Under component-wise encoding, the value is stored as a tagged pair: either $(\mathtt{L}, \mathrm{enc}_A(a, k))$ or $(\mathtt{R}, \mathrm{enc}_B(b, k))$, where the tag $\mathtt{L}$ or $\mathtt{R}$ is in the clear (or distinguishable by the encoding structure—for instance, $A$-cipher-values and $B$-cipher-values occupy disjoint regions of bit-string space, or carry a type identifier). The untrusted machine reads the tag and routes to the correct handler. Pattern matching is available.

But the tag is observable. Given a cipher value, the untrusted machine learns whether it encodes an $A$-value or a $B$-value. For any distribution $D$ on $A + B$, the fraction of queries that are $A$-values is observable, leaking $\Pr_{v \sim D}[v \in A]$.

**No intermediate encoding achieves both.** Suppose an encoding achieves both tag hiding (advantage $\leq \delta'$ for some small $\delta'$) and untrusted pattern matching (success probability $\geq 1 - \gamma$). Pattern matching requires a function $\tau : \{0, 1\}^n \to \{A, B\}$ that is correct on valid encodings with probability $\geq 1 - \gamma$. Tag hiding requires that no such function exists with advantage better than $\delta'$. Let $p = \Pr[v \in A]$ be the prior probability of an $A$-value. A trivial strategy that ignores the encoding and always guesses the majority type achieves accuracy $\max(p, 1 - p)$. The advantage of $\tau$ over this baseline is at least $(1 - \gamma) - \max(p, 1 - p)$. Tag hiding requires this advantage to be at most $\delta'$. Therefore $(1 - \gamma) - \max(p, 1 - p) \leq \delta'$, which means either $\gamma \geq 1 - \max(p, 1 - p) - \delta'$ (pattern matching fails often) or $\delta' \geq (1 - \gamma) - \max(p, 1 - p)$ (the tag is substantially leaked). For the balanced case $p = 1/2$, this simplifies to $1/2 - \gamma \leq \delta'$. Unbalanced distributions ($p \neq 1/2$) make the impossibility stronger: the baseline accuracy is higher, leaving less room for the encoding to add information without leaking the tag. □

*Remark* 4.1 (Contrast with products). Products leak correlations; sums leak the tag. The asymmetry is structural. For a product $A \times B$, the two components are always both present; the question is whether their relationship (correlation) is visible. For a sum $A + B$, only one component is present;

the question is whether which one is visible. A product hides *what values co-occur*; a sum hides *what type is occupied*.

*Remark* 4.2 (Trusted pattern matching). Under joint encoding $\mathsf{C}(A + B)$, pattern matching is still possible—by the trusted machine. The trusted machine decodes $\mathrm{dec}(c)$, inspects the tag, applies the appropriate handler in plaintext, and re-encodes the result. This requires a round trip through the trusted machine for every case analysis. The cost is a communication round, not an impossibility. The impossibility is for *untrusted* pattern matching without the trapdoor.

*Example* 4.1 (Cipher optionals). The optional type $\mathrm{Maybe}(A) = \mathbf{1} + A$ is a sum. Under $\mathsf{C}(\mathbf{1} + A)$, the untrusted machine cannot tell whether a cipher value encodes Nothing or $\mathrm{Just}(a)$. Under $\mathsf{C}(\mathbf{1}) + \mathsf{C}(A)$, the untrusted machine knows whether the value is present, but not what it is. For a database application, the first encoding hides which records have null fields; the second reveals the null pattern but permits null-checking without the trapdoor.

### 4.4 Exponential Types

The exponential type $B^A = A \to B$ is the type of functions from $A$ to $B$. A cipher map $\hat{f}$ for $f : A \to B$ is already an element of the cipher exponential type: it is a total function on $\{0,1\}^n$ that, when restricted to valid encodings of $A$, produces (approximate) encodings of $B$-values.

This observation, developed in [9], means the cipher exponential type is not a new construction but the cipher map abstraction itself. The four properties (totality, representation uniformity, correctness, composability) are exactly the conditions that make $\hat{f}$ a well-behaved element of $\mathsf{C}(A \to B)$.

The encoding granularity trade-off from Section 4.2 extends to exponentials. Given cipher maps $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ and $\hat{g} : \{0,1\}^n \to \{0,1\}^n$, applying them independently (component-wise) to the two components of a product cipher value reveals that both functions are being applied—leaking the computation pattern. Encoding the pair of function applications as a single cipher map $\widehat{(f,g)} : \{0,1\}^n \to \{0,1\}^n$ for the product function $(f,g)(a,b) = (f(a), g(b))$ hides the computation pattern but requires a dedicated construction.

## 5 Orbit Closure and Information Leakage

The type constructors of Section 4 describe static confidentiality properties: what the untrusted machine can learn from the structure of a single cipher value. We now develop a tool for analyzing dynamic confidentiality: what the untrusted machine can learn by actively applying the cipher maps it holds.

### 5.1 Definition

**Definition 5.1** (Orbit closure)**.** Let $F = \{\hat{f}_1, \ldots, \hat{f}_k\}$ be a set of cipher maps (total functions $\{0,1\}^n \to \{0,1\}^n$) available to the untrusted machine, and let $c \in \{0,1\}^n$ be a cipher value. The *orbit closure* of $c$ under $F$ is the smallest set $\mathrm{orbit}_F(c) \subseteq \{0,1\}^n$ such that:

1. $c \in \mathrm{orbit}_F(c)$, and
2. if $c' \in \mathrm{orbit}_F(c)$ and $\hat{f}_i \in F$, then $\hat{f}_i(c') \in \mathrm{orbit}_F(c)$.

Equivalently, $\mathrm{orbit}_F(c)$ is the set of all bit strings reachable from $c$ by composing operations in $F$ in any order and any number of times.

Since $\{0,1\}^n$ is finite, $\mathrm{orbit}_F(c)$ is always finite and can be computed by iterative expansion (breadth-first search from $c$, applying each $\hat{f}_i$ at each step, until no new values appear).

*Remark* 5.1. The orbit closure depends only on the cipher maps $F$ available to the untrusted machine and the starting cipher value $c$. It does not depend on the latent function, the domain, or the trapdoor. This makes it a tool for analysis by the trusted machine designer: given the operations you plan to expose, how much can the untrusted machine explore?

## 5.2 Monotonicity

**Theorem 5.1** (Monotonicity of orbit closure)**.** *If $F \subseteq G$ (every operation in $F$ is also in $G$), then* $\mathrm{orbit}_F(c) \subseteq \mathrm{orbit}_G(c)$ *for all $c \in \{0,1\}^n$.*

*Proof.* By induction on the iterative construction of $\mathrm{orbit}_F(c)$.

*Base case.* $c \in \mathrm{orbit}_F(c)$ and $c \in \mathrm{orbit}_G(c)$ by definition.

*Inductive step.* Suppose $c' \in \mathrm{orbit}_F(c)$ and $c' \in \mathrm{orbit}_G(c)$ (inductive hypothesis). For any $\hat{f}_i \in F$, we have $\hat{f}_i(c') \in \mathrm{orbit}_F(c)$ by Definition 5.1. Since $F \subseteq G$, $\hat{f}_i \in G$, so $\hat{f}_i(c') \in \mathrm{orbit}_G(c)$ by Definition 5.1 applied to $G$.

Every element of $\mathrm{orbit}_F(c)$ is therefore also in $\mathrm{orbit}_G(c)$. □

The interpretation is direct: exposing more operations to the untrusted machine can only enlarge (never shrink) the set of cipher values it can reach. Each additional operation is an additional edge in the reachability graph over $\{0,1\}^n$.

**Corollary 5.2.** $\mathrm{orbit}_\emptyset(c) = \{c\}$ *for all $c$. If the untrusted machine has no cipher maps, it learns nothing beyond the cipher value it was given.*

*Proof.* With $F = \emptyset$, only the base case of the orbit definition applies. □

## 5.3 Confidentiality Bound

The orbit closure quantifies reachability. We now connect it to confidentiality. The key idea: the untrusted machine knows, without the trapdoor, that the latent value encoded by $c$ must be consistent with every cipher value in $\mathrm{orbit}_F(c)$. A larger orbit means more of the cipher space is determined by $c$, leaving fewer possibilities for the latent value.

**Definition 5.2** (Candidate set)**.** Given a cipher value $c$, operations $F$, and the orbit $\mathrm{orbit}_F(c)$, the *candidate set* is the set of latent values consistent with the observed orbit:

$$\mathrm{Cand}_F(c) = \{x \in X \mid \exists\, k \text{ s.t. } \mathrm{enc}(x,k) = c \text{ and } \mathrm{orbit}_F(\mathrm{enc}(x,k)) = \mathrm{orbit}_F(c)\}.$$

That is, $x$ is a candidate if encoding $x$ produces $c$ and the orbit structure starting from $c$ matches what the adversary observes.

**Definition 5.3** (Confidentiality)**.** The *confidentiality* of a cipher value $c$ under operations $F$ is the adversary's remaining uncertainty about the latent value:

$$\mathrm{conf}_F(c) = 1 - \frac{|\mathrm{Cand}_F(c)|}{|X|}.$$

When $\mathrm{conf}_F(c) = 1$, the adversary has learned nothing. When $\mathrm{conf}_F(c) = 0$, the latent value is uniquely determined.

**Theorem 5.3** (Confidentiality bound)**.** *Let $F$ be a set of cipher maps available to the untrusted machine, $c$ a cipher value, and assume representation uniformity with parameter $\delta \approx 0$. Then:*

$$\mathrm{conf}_F(c) \geq 1 - \frac{|\mathrm{orbit}_F(c)|}{2^n}.$$

*Proof.* Under $\delta \approx 0$, encodings are spread approximately uniformly across $\{0,1\}^n$. The orbit $\mathrm{orbit}_F(c)$ is a set of bit strings that the adversary can compute but not decode. In the worst case, every orbit element is a valid encoding of a distinct latent value. The adversary then knows the latent value is one of at most $|\mathrm{orbit}_F(c)|$ possibilities out of $|X|$ total. Since $|X| \leq 2^n$ (at most $2^n$ distinct encodings), the fraction of the cipher space explored is $|\mathrm{orbit}_F(c)|/2^n$, and the remaining unexplored fraction bounds the confidentiality.

This bound is loose: it ignores the functional relationships between orbit elements (e.g., the adversary knows that orbit element $c' = \hat{f}(c)$, which constrains the possible latent values beyond mere set size). Tighter bounds for specific operations are an open problem. □

*Remark* 5.2. When $|\mathrm{orbit}_F(c)| = 2^n$ (the entire cipher space is reachable), the confidentiality bound is zero: the adversary can potentially distinguish every latent value by exploring the orbit structure. When $|\mathrm{orbit}_F(c)| = 1$ (no operations available), confidentiality is $1 - 2^{-n} \approx 1$.

## 5.4 Examples

*Example* 5.1 (Boolean operations). Let $X = \{0,1\}$ (Boolean) with cipher maps $\widehat{\mathrm{AND}}$ and $\widehat{\mathrm{NOT}}$ available to the untrusted machine. Given a cipher value $c$ encoding some unknown Boolean $x$, the untrusted machine can compute:

$$\begin{aligned} c_1 &= \widehat{\mathrm{NOT}}(c), \\ c_2 &= \widehat{\mathrm{AND}}(c, c_1) = \widehat{\mathrm{AND}}(c, \widehat{\mathrm{NOT}}(c)). \end{aligned}$$

Now, $\mathrm{AND}(x, \mathrm{NOT}(x)) = 0$ for all $x$ (up to correctness $\eta$). So $c_2$ is an (approximate) cipher encoding of the constant FALSE. If the untrusted machine has a reference cipher value $c_{\mathrm{F}}$ known to encode FALSE (for instance, from a prior computation $\widehat{\mathrm{AND}}(c', \widehat{\mathrm{NOT}}(c'))$ for some other cipher value $c'$), it can compare $c_2$ with $c_{\mathrm{F}}$.

This comparison reveals information: if $c_2 = c_{\mathrm{F}}$ (same representation), the untrusted machine confirms the tautology and potentially identifies the representation index. Writing $F_0 = \{\widehat{\mathrm{AND}}, \widehat{\mathrm{NOT}}\}$, the orbit $\mathrm{orbit}_{F_0}(c)$ includes $c$, $c_1$, $c_2$, and all further compositions—quickly covering a nontrivial fraction of the cipher space for small $n$.

*Example* 5.2 (Successor on a bounded type). Let $X = \{0, 1, \ldots, m-1\}$ (integers mod $m$) with a single cipher map $\widehat{\mathrm{succ}}$ implementing the successor function $x \mapsto (x+1) \bmod m$. Given any cipher value $c$ encoding some $x$:

$$\mathrm{orbit}_{\{\widehat{\mathrm{succ}}\}}(c) = \{c,\ \widehat{\mathrm{succ}}(c),\ \widehat{\mathrm{succ}}^2(c),\ \ldots,\ \widehat{\mathrm{succ}}^{m-1}(c)\}.$$

After $m$ applications, the orbit returns to $c$ (since successor is cyclic on $\mathbb{Z}/m\mathbb{Z}$). The orbit has exactly $m$ elements (assuming $\eta = 0$). The untrusted machine does not learn which element is $\mathrm{enc}(0, k)$, but it learns the cycle structure: there exists a cycle of length $m$ starting from $c$. This reveals $m = |X|$, the domain cardinality.

By Theorem 5.3, the confidentiality is $1 - m/2^n$. For $m \ll 2^n$, this is close to 1: the orbit covers a tiny fraction of the cipher space. For $m$ close to $2^n$, confidentiality degrades.

*Example* 5.3 (Branching). Suppose the untrusted machine holds cipher maps $\hat{f}$ and $\hat{g}$ and evaluates both on a cipher value $c$, obtaining $\hat{f}(c)$ and $\hat{g}(c)$. The orbit of $c$ under $\{\hat{f}, \hat{g}\}$ includes both results plus all further compositions. If $\hat{f}$ and $\hat{g}$ are the two branches of a conditional (the "then" and "else" handlers), both cipher results are visible to the untrusted machine—even though in plaintext computing, only one branch would be taken.

This is a form of information leakage specific to trapdoor computing: the untrusted machine does not know which branch is "correct," but it sees both outcomes, enlarging the orbit and reducing confidentiality. Mitigating this requires either (a) evaluating both branches and having the trusted machine select the correct result, or (b) encoding the branch selection as part of the cipher map (a joint encoding of the conditional). Strategy (a) doubles the computation; strategy (b) requires a dedicated cipher map construction.

## 5.5 Typed Composition Chains

The orbit grows because the adversary can feed cipher map outputs back as inputs, composing operations to reach new cipher values. A direct mitigation: *prevent self-composition by typing the cipher spaces.*

**Definition 5.4** (Typed composition chain)**.** A *typed composition chain* of depth $k$ is a sequence of cipher maps $\hat{f}_0, \hat{f}_1, \ldots, \hat{f}_{k-1}$ where $\hat{f}_i : \mathsf{C}(A)_i \to \mathsf{C}(A)_{i+1}$ and the cipher spaces $\mathsf{C}(A)_0, \mathsf{C}(A)_1, \ldots, \mathsf{C}(A)_k$ are *distinct* (disjoint hash value sets, different secrets).

Since $\mathsf{C}(A)_i \neq \mathsf{C}(A)_j$ for $i \neq j$, the output of $\hat{f}_i$ cannot be fed into $\hat{f}_j$ for $j \neq i+1$. The adversary can compose at most $k$ operations in sequence, and cannot loop.

**Proposition 5.4** (Orbit bound for typed chains)**.** *For a typed composition chain of depth $k$ with cipher spaces of sizes $|\mathsf{C}(A)_0|, \ldots, |\mathsf{C}(A)_k|$, the orbit of any cipher value $c \in \mathsf{C}(A)_0$ is bounded by*

$$|\mathrm{orbit}_F(c)| \leq 1 + k,$$

*where each step adds at most one new cipher value (the output of $\hat{f}_i$ applied to the previous result). The confidentiality satisfies $\mathrm{conf}_F(c) \geq 1 - (1+k)/2^n$.*

*Proof.* Starting from $c \in \mathsf{C}(A)_0$, the adversary computes $c_1 = \hat{f}_0(c) \in \mathsf{C}(A)_1$, $c_2 = \hat{f}_1(c_1) \in \mathsf{C}(A)_2$, and so on up to $c_k = \hat{f}_{k-1}(c_{k-1}) \in \mathsf{C}(A)_k$. No operation maps back into an earlier cipher space, so the orbit is exactly $\{c, c_1, \ldots, c_k\}$ with $|\mathrm{orbit}_F(c)| = 1 + k$. The confidentiality bound follows from Theorem 5.3. □

*Example* 5.4 (Depth-limited Boolean search). For Boolean search with maximum query depth 2 (e.g., $w_1 \wedge w_2$ but not $w_1 \wedge w_2 \wedge w_3$), define:

- Cipher sets output to $\mathsf{C}(\mathrm{Bool})_0$.
- $\mathrm{AND}_0 : \mathsf{C}(\mathrm{Bool})_0 \times \mathsf{C}(\mathrm{Bool})_0 \to \mathsf{C}(\mathrm{Bool})_1$.
- $\mathrm{AND}_1 : \mathsf{C}(\mathrm{Bool})_1 \times \mathsf{C}(\mathrm{Bool})_1 \to \mathsf{C}(\mathrm{Bool})_2$.
- No $\mathrm{AND}_2$ is defined.

The adversary can compose at most two AND operations. The orbit from any cipher Boolean value has size at most 3. Contrast with a single shared cipher Boolean space, where the adversary can self-compose indefinitely and the orbit grows without bound.

The general principle: **the type system controls the orbit**. By choosing how many cipher space levels to define, the system designer sets the maximum composition depth and therefore the maximum orbit size and minimum confidentiality. This is a compile-time (construction-time) decision with no runtime cost: the untrusted machine simply does not have cipher maps for deeper levels.

# 6 Cipher Turing Machine

The cipher map abstraction represents a function $f : X \to Y$ as a lookup table: a total function $\hat{f} : \{0,1\}^n \to \{0,1\}^n$ that maps each cipher encoding of $x$ to a cipher encoding of $f(x)$. The space complexity is $O(|X|)$—the table must have an entry for every element of the domain (plus noise entries). For large domains (e.g., string processing over an alphabet of size $|\Sigma|$ with strings of length $\ell$, giving $|X| = |\Sigma|^\ell$), this is prohibitive.

A Turing machine provides an alternative computational model whose space depends on the *program* (states and tape alphabet), not the *data* (input size). We define a cipher Turing machine and analyze its space and leakage properties.

## 6.1 Definition

**Definition 6.1** (Cipher Turing machine)**.** A *cipher Turing machine* is a tuple $(Q, \Gamma, \delta_T, q_0, q_{\text{halt}})$ together with a cipher encoding of the transition function, where:

- $Q$ is a finite set of states, $\Gamma$ is a finite tape alphabet (with a distinguished blank symbol $\sqcup$).
- The transition function $\delta_T : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$ maps (current state, symbol read) to (next state, symbol to write, head direction).
- $\delta_T$ is implemented as a cipher map $\hat{\delta}_T : \{0,1\}^n \to \{0,1\}^n$, where the input is a cipher encoding of a (state, symbol) pair and the output is a cipher encoding of a (state, symbol, direction) triple.
- The tape is a sequence of cipher values, each encoding a symbol in $\Gamma$.
- $q_0$ is the initial state (cipher-encoded); $q_{\text{halt}}$ is the halting state.

Execution proceeds as follows. At each step, the untrusted machine reads the cipher symbol $c_{\text{sym}}$ at the current head position and the current cipher state $c_q$. It evaluates $\hat{\delta}_T$ on $\text{enc}((q, \gamma), k)$ where $q$ and $\gamma$ are the values encoded by $c_q$ and $c_{\text{sym}}$—but since the untrusted machine cannot decode, it instead evaluates $\hat{\delta}_T$ on the *pair cipher value* $\text{enc}_{\text{pair}}(c_q, c_{\text{sym}})$, which requires a cipher map for the pairing operation. The output is decoded (by the cipher map structure, not by the untrusted machine) into a cipher state, a cipher symbol (written to the tape), and a head direction.

The head direction $\{L, R, S\}$ is **not** cipher-encoded: it is in the clear. This is the fundamental leakage of the cipher Turing machine.

*Remark* 6.1. The head direction must be in the clear because the untrusted machine needs to know which tape cell to access next. Without knowing the direction, it cannot update the head position. Ciphering the direction would require the untrusted machine to access all three possible next cells (left, right, stay) on every step, tripling the work per step and still revealing that a step occurred.

## 6.2 Space Complexity

**Proposition 6.1** (Cipher TM space complexity)**.** *The cipher map $\hat{\delta}_T$ for a cipher Turing machine with $|Q|$ states and $|\Gamma|$ tape symbols has space complexity $O(|Q| \cdot |\Gamma|)$, independent of the input domain size $|X|$.*

*Proof.* The transition function $\delta_T : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$ has domain $Q \times \Gamma$ of size $|Q| \cdot |\Gamma|$. The cipher map $\hat{\delta}_T$ is a cipher map for $\delta_T$, which by [9, Sec. 6] has space proportional to its domain size: $O(|Q| \cdot |\Gamma|)$ entries (plus the $-\log_2 \varepsilon$ bits per entry for noise budget and the $H(Q \times \Gamma \times \{L, R, S\})$ bits per entry for value encoding).

In contrast, a cipher map $\hat{f}$ for an arbitrary function $f : X \to Y$ has space $O(|X|)$. When the computation is specified as a Turing machine program rather than a lookup table, $|Q| \cdot |\Gamma|$ replaces $|X|$ as the domain size. For large input domains ($|X| \gg |Q| \cdot |\Gamma|$), this is a qualitative improvement: the cipher map's size depends on the program complexity, not the data size.

The tape itself requires space proportional to the number of tape cells used during computation (at most the running time), but each tape cell is a single cipher value encoding a symbol in $\Gamma$—the per-cell cost is $O(1)$ cipher values, independent of $|X|$. □

*Remark* 6.2. The space saving comes with a time cost: where a cipher map evaluates $f(x)$ in $O(1)$ cipher map lookups (two hash evaluations for the two-level construction [9]), a cipher Turing machine requires as many steps as the Turing machine takes to compute $f(x)$. The trade-off is space (lookup table vs. program) for time ($O(1)$ vs. $O(T(|x|))$ where $T$ is the running time).

## 6.3 Head Movement Leakage

The cipher Turing machine's head direction is in the clear. This means the untrusted machine observes the sequence of head movements $d_1, d_2, d_3, \ldots \in \{L, R, S\}^*$ for every computation.

**Proposition 6.2** (Head movement determinism)**.** *For a deterministic cipher Turing machine with correctness $\eta = 0$, the head movement sequence is a deterministic function of the input. Two inputs that produce different head movement sequences are distinguishable by the untrusted machine.*

*Proof.* When $\eta = 0$, the cipher map $\hat{\delta}_T$ is exact: every transition produces the correct next state, symbol, and direction. The head movement sequence is therefore identical to the sequence produced by the plaintext Turing machine on the same input. Since the plaintext Turing machine is deterministic, the head movement sequence is a deterministic function of the input.

If $x_1 \neq x_2$ produce different head movement sequences, i.e., $(d_1^{(1)}, d_2^{(1)}, \ldots) \neq (d_1^{(2)}, d_2^{(2)}, \ldots)$, the untrusted machine observes different sequences and can distinguish the two inputs (or at least partition inputs into equivalence classes by their head movement pattern). □

*Remark* 6.3 (Comparison with ORAM)*.* Oblivious RAM [5] is designed precisely to hide access patterns: the sequence of memory addresses accessed is independent of the input. A cipher Turing machine makes no such guarantee—the head movement pattern is visible. The cipher TM's confidentiality comes from a different source: the tape contents (cipher symbols) are opaque, and the transition function (cipher map) is opaque. The adversary sees which cells are visited but not what is read or written. This is weaker than ORAM but requires no $O(\log N)$ overhead per access.

*Remark* 6.4 (Nonzero $\eta$)*.* When $\eta > 0$, the cipher transition function occasionally produces incorrect results. An incorrect transition may produce a different head direction than the correct one, introducing noise into the head movement sequence. This provides a weak form of access-pattern obfuscation—the observed sequence differs from the true sequence with probability depending on $\eta$

and the number of steps—but it is unreliable as a privacy mechanism because errors are systematic (the same elements always fail for a given seed), not random.

# 7 Discussion

## 7.1 Cipher Program Construction

The type-theoretic trade-offs of Sections 4.2 and 4.3 have a direct operational realization: given a program (an expression tree), which subtrees should be replaced by cipher maps?

**Definition 7.1** (Cipher node annotation)**.** Given a program $P$ represented as an expression tree, a *cipher node annotation* marks a subset of nodes $\{n_1, \ldots, n_k\}$ in the tree. Each marked node $n_i$, together with its entire subtree, is replaced by a cipher map: the subtree is evaluated at construction time (on the trusted machine) over a specified domain, and the resulting mapping is stored as a PHF-backed cipher map. Unmarked nodes above the cipher nodes become a *combiner*: itself a cipher map built over the hash-value outputs of the components.

The placement of cipher node annotations controls the encoding granularity:

- **Annotation at the root**: the entire program is one cipher map. Maximum confidentiality (no intermediate values exposed), maximum space ($O(|X|)$ where $X$ is the input domain of the full program).
- **Annotation at the leaves**: each leaf function is a separate cipher map. The combiner chains them. Intermediate cipher values are visible to the untrusted machine, leaking correlations between components (as in the product-type trade-off of Section 4.2).
- **Annotation at intermediate nodes**: a spectrum between the two extremes. Deeper annotations absorb more computation into cipher maps (more confidential, more space); shallower annotations expose more structure (less space, more leakage).

*Remark* 7.1 (Shared variables and compositional leakage)*.* When two cipher nodes $f$ and $g$ share an input variable $x$, the untrusted machine observes two cipher values $\mathrm{enc}_f(x, k_f)$ and $\mathrm{enc}_g(x, k_g)$ derived from the same latent $x$. Even if both encodings are individually $\delta$-close to uniform, the joint distribution of $(\mathrm{enc}_f(x), \mathrm{enc}_g(x))$ preserves the correlation (as in [9, Prop. 9.1]). The only mitigation is to merge $f$ and $g$ into a single cipher node (eliminating the shared intermediate) or to inject noise queries that dilute the correlation signal.

**Implementation via tracing.** A practical realization uses Python function decorators and runtime tracing. Functions marked with a `@cipher_node` decorator are intercepted during a tracing pass over the input domain. The tracing records which cipher nodes are called, with what arguments, and what they return. From this trace, the system:

1. builds a cipher map for each traced cipher node,
2. builds a combiner cipher map over the hash-output tuples of the components,
3. infers the input routing (which program arguments feed which cipher node) from the traced call patterns.

Short-circuit evaluation (e.g., Python's `x if cond else y`) is handled by computing all component cipher maps for every input, even when the original program skips some branches. The combiner's domain includes hash tuples for both branches. The resulting composed cipher program evaluates entirely on the untrusted machine with no round trips to the trusted machine.

**Relationship to cipher maps.** This paper extends the cipher map framework of [9] in two directions: inward (what happens when cipher values carry algebraic structure) and outward (what happens when the untrusted machine actively explores the cipher space). The cipher map paper focuses on individual maps and their four properties; this paper studies how those properties interact with type constructors and how confidentiality degrades under active use.

**Relationship to Bernoulli data types.** The accuracy-side analysis of algebraic types over approximate values is developed in [8], which studies the Kronecker factorization of joint confusion matrices for product types and the error propagation through sum-type constructors. That paper asks: how does $\eta$ change when types are composed? This paper asks the complementary question: how does confidentiality change when types are composed? Together, they characterize the full trade-off space (accuracy $\times$ confidentiality) for algebraic types over trapdoor computing.

**The encoding granularity principle.** The product and sum trade-offs in Sections 4.2 and 4.3 are instances of a single principle developed in [9, Sec. 9]: coarser encoding granularity (encoding more values as a single unit) yields better confidentiality but fewer operations available to the untrusted machine. The entanglement parameter $p$ [9, Sec. 9.3] controls the spectrum: $p = 1$ (component-wise) gives maximum functionality and minimum confidentiality; $p = k$ (whole-state) gives maximum confidentiality and minimum functionality.

## 7.2 Cipher Boolean Algebra

The cipher Boolean type $\mathsf{C}(\text{Bool})$ illustrates how the sum-type trade-off specializes to the most fundamental algebraic structure.

**Definition 7.2** (Cipher Boolean type)**.** A cipher Boolean type over $\{0,1\}^n$ is a partition of the hash space into three disjoint regions:

- $T \subset \{0,1\}^n$: the True region ($|T|/2^n = p_{\text{true}}$),
- $F \subset \{0,1\}^n$: the False region ($|F|/2^n = p_{\text{false}}$),
- $N = \{0,1\}^n \setminus (T \cup F)$: the noise region.

The partition is derived from a secret via a pseudorandom permutation. Without the secret, membership in $T$, $F$, or $N$ is indistinguishable from random.

The noise region $N$ plays a structural role beyond mere error tolerance.

**Proposition 7.1** (Noise unreliability)**.** *Let* AND*,* OR*,* NOT *be cipher maps implementing Boolean operations over the cipher Boolean type. If any input is a noise value $n \in N$, the output is* uncorrelated *with the intended Boolean result: it lands in $T$, $F$, or $N$ according to the allocation ratios $p_T$, $p_F$, $p_N$, independent of the other input's value.*

*Proof.* The AND, OR, and NOT cipher maps are constructed over the domain $T \cup F$. An input from $N$ is outside this domain. Since the cipher map is total (backed by a PHF with random noise in unused slots), a noise input maps to a random slot whose contents decode to a value in $T$ with probability $p_T$, in $F$ with probability $p_F$, and in $N$ with probability $p_N$. For the standard allocation ($p_T = 0.05$, $p_F = 0.90$, $p_N = 0.05$), a noise input produces False 90% of the time, True 5%, and noise 5%. Crucially, this output is independent of the other (valid) input: the cipher map cannot "correct" for a noise input because it does not know which inputs are noise. □

*Remark* 7.2 (Noise prevents output certainty). The noise region ensures that an adversary observing a sequence of Boolean cipher map evaluations cannot achieve certainty about any individual result. Consider an adversary that repeatedly ANDs a cipher value $c$ with random cipher values $r_1, r_2, \ldots$. If $c \in T$ (True), then $\mathrm{AND}(c, r_i)$ produces True when $r_i \in T$ and False when $r_i \in F$. But when $r_i \in N$ (5% probability), the output is noise, which the adversary cannot distinguish from a legitimate False. After $k$ ANDs, the adversary has seen some True and some False/noise results, but cannot determine with certainty whether any given False result is real or noise-induced.

Dually, an adversary that repeatedly ORs $c$ with random values sees True for $c \in T$ (always True), but again noise inputs produce unpredictable outputs. For $c \in F$, the OR produces True only when $r_i \in T$ (5%), False when $r_i \in F$ (90%), and noise when $r_i \in N$ (5%). The noise fraction creates a floor of uncertainty in any statistical test the adversary performs.

*Remark* 7.3 (Generalization beyond Bool). Definition 7.2 generalizes to any finite type $Y$. A cipher type $\mathsf{C}(Y)$ partitions $\{0,1\}^n$ into $|Y|$ value regions plus a noise region, with $|R_y|/2^n \propto \Pr[Y = y]$ (Shannon-optimal allocation). Operations $f : Y_1 \times \cdots \times Y_k \to Z$ become cipher maps over the product of input regions. The cipher Boolean is the special case $Y = \{\mathrm{True}, \mathrm{False}\}$; the same construction applies to enumerated types, bounded integers, or any codomain where the output distribution is known.

**Practical allocation.** The cipher Boolean allocation $|T|/2^n = 0.05$, $|F|/2^n = 0.90$, $|N|/2^n = 0.05$ is motivated by the set membership use case. A cipher set $\hat{1}_A : \mathsf{C}(X) \to \mathsf{C}(\mathrm{Bool})$ maps domain elements to cipher Booleans. Members are forced into $T$ by the construction. Non-members land randomly: 90% decode as False (correct rejection), 5% as True (false positive), 5% as noise. The false positive rate equals $p_{\mathrm{true}}$ and is a system parameter, not dependent on the set size or universe.

**Composition with set operations.** Boolean search queries like "$w_1 \wedge w_2 \wedge \neg w_3$" are evaluated entirely on the untrusted machine by chaining cipher set evaluations through cipher AND/OR/NOT maps. Each Boolean operation is itself a cipher map (constructed via PHF over the $|T \cup F|^2$ input pairs for AND/OR, or $|T \cup F|$ for NOT). The untrusted machine sees opaque hash values flowing through opaque total functions at every stage.

**Experimental results.** A reference implementation (`cipher-maps`, Python with PHF backend via `phobic`) validates the construction on the 20 Newsgroups corpus (18,266 documents, 58,903 unique words). With $n = 8$ bits, $|T| = 13$, $|F| = 230$, the AND and OR cipher maps each have $(13 + 230)^2 = 59{,}049$ entries.

*Boolean search at scale.* Table 1 shows precision and recall for representative Boolean queries over 5,000 documents, with all Boolean operations evaluated on the untrusted machine. Construction rate: 843 documents per second.

Table 1: Boolean encrypted search (5,000 documents, $p_T = 0.05$).

| Query | Precision | Recall | FP | Time |
|---|---|---|---|---|
| Single term | 0.39 | 1.00 | 248 | 0.10s |
| 2-term AND | 0.26 | 1.00 | 42 | 0.30s |
| 3-term AND | 0.33 | 1.00 | 12 | 0.60s |
| OR | 0.37 | 0.97 | 428 | 0.30s |
| OR AND NOT | 0.33 | 0.88 | 389 | 0.55s |

AND queries maintain perfect recall while false positives decrease (248 for single term, 12 for 3-term AND). OR and NOT queries lose recall (0.97 and 0.88) due to noise propagation: when a cipher set returns a noise value $n \in N$, the subsequent OR or NOT operation produces noise or an unpredictable result, effectively dropping the document from the result set.

*FPR compounding vs. Bernoulli model.* The Bernoulli composition model [8] predicts $\mathrm{FPR}_{\mathrm{AND},k} = p_T^k$ for $k$ independent AND operations. Empirically, the OR chain matches theory closely (ratio $\approx 0.9$ across chain lengths 1–5), but the AND chain diverges at depth $> 2$: the empirical FPR exceeds the predicted $p_T^k$ by an order of magnitude at $k = 3$. The reason: the Bernoulli model assumes each test is an independent draw, but the cipher Boolean AND is a deterministic cipher map. After one AND, the result is a specific cipher Boolean value (not a fresh random draw), and feeding it into the next AND produces a deterministic output. The independence assumption breaks at the cipher map level even though it holds at the element level within a single cipher set.

*Encoding granularity.* We built a 7-function loan approval pipeline at three granularity levels:

| Level | Build time | Space | Intermediates |
|---|---|---|---|
| Root (1 cipher map) | 0.009s | 694 B | 0 |
| Intermediate (3 groups) | 6.8s | — | 3 |
| Leaf (7 nodes) | 5.9s | — | 7 |

All three produce zero errors on the full domain (150 inputs). The root annotation is 650× faster (PHF backend vs. seed search) and exposes zero intermediate cipher values. The trade-off: the root annotation requires the entire input domain at construction time, while the leaf annotation builds smaller cipher maps that compose dynamically.

**Open questions.**

1. *Recursive types.* Lists, trees, and other recursive types are built from sums and products. The sum-type impossibility (Theorem 4.2) implies that recursive traversal (which requires pattern matching at each node) either leaks the structure (which constructors are used at each level) or requires a trusted-machine round trip at each node. Quantifying the total leakage for a recursive traversal of depth $d$ is open.

2. *Tight orbit bounds for specific constructions.* Theorem 5.3 gives a general upper bound on confidentiality in terms of orbit size. For specific cipher map constructions (e.g., the trapdoor boolean algebra of [9]), tighter bounds may be possible by exploiting the structure of the construction.

3. *Cipher TM with oblivious head.* Can the head direction be ciphered at acceptable cost? The naive approach (evaluate all three possible next states) triples the work per step. Whether a sub-multiplicative overhead exists, achieving partial head-movement hiding without full ORAM cost, is open.

4. *Partial sum-type hiding.* Theorem 4.2 shows that full tag hiding and full untrusted pattern matching are mutually exclusive. Is there a meaningful intermediate notion—hiding the tag from some adversary classes while permitting restricted pattern matching? For instance, one could imagine a construction where the tag is hidden against passive adversaries (who only observe cipher values) but available to active adversaries (who evaluate cipher maps).

## 8 Conclusion

We have shown that algebraic type constructors create unavoidable confidentiality costs in trapdoor computing. The sum-type impossibility theorem establishes that tag hiding and untrusted pattern matching are mutually exclusive for any encoding. The orbit closure framework provides a quantitative bound on confidentiality degradation as the untrusted machine applies operations. These results apply to any cipher map construction, not just specific implementations.

On the practical side, cipher Boolean types with AND/OR/NOT as cipher maps enable Boolean search queries evaluated entirely on the untrusted machine. Experiments on the 20 Newsgroups corpus (5,000 documents) show perfect recall for AND queries, with false positive rates matching theoretical predictions for single-term queries and compounding predictably through Boolean chains. The encoding granularity spectrum is measurable: root annotation (one cipher map for the entire program) is 650× faster and exposes zero intermediates compared to leaf annotation (one cipher map per function).

The gap between the Bernoulli independence model and the cipher map implementation for AND chain composition points to future work on tighter error models for composed cipher maps.

## Acknowledgments

The algebraic cipher types concept originates in a 2019–2022 C++ notebook by the author, which explored cipher Booleans, cipher unions, and cipher Turing machines through concrete implementations. The present paper extracts and formalizes the theoretical content from that notebook, building on the cipher map framework developed in [9].

## References

[1] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.

[2] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In *Theory of Cryptography Conference*, pages 253–273, 2011.

[3] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security*, pages 79–88, 2006.

[4] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, pages 169–178, 2009.

[5] Oded Goldreich and Rafail Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, 1996.

[6] Muhammad Naveed, Seny Kamara, and Charles V Wright. Inference attacks on property-preserving encrypted databases. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security*, pages 644–655, 2015.

[7] Andrei Sabelfeld and Andrew C Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[8] Alexander Towell. Bernoulli data types: Approximate algebraic types with quantitative error, 2026. Manuscript in preparation.

[9] Alexander Towell. Cipher maps: Total functions as trapdoor approximations, 2026. Manuscript in preparation.

[10] Andrew C Yao. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science*, pages 160–164, 1982.